

Agentic AI: Why It Matters, and How to Prove It Works

What an agent-friendly storefront actually buys you, and the exact checks that tell you whether this month's work is live, correct and earning its keep.

DATE	APPLIES TO	COMPANION TO
11 August 2026	Solana · BBQ · OKO	Development Update, 5–11 Aug

Contents

- 1. Part One — What "agentic AI friendly" means, and what it is worth
- 2. Part Two — How to test it, in five levels
- 3. Traps that make a passing test lie
- 4. How to tell, over months, whether it was worth it

Part One — What it means and what it is worth

Shopping is starting to happen one step removed from the website. Instead of searching, clicking three results and comparing tabs, a growing number of buyers ask an assistant — ChatGPT, Perplexity, Claude, Google's AI results — a question like *"a built-in 4-burner gas grill under \$3,000 that fits a 34-inch cutout"*. The assistant does the fetching and comparing, and returns a short list.

The store never sees that comparison happen. It either appears in the answer or it does not. "Agentic AI friendly" simply means being the kind of site that can appear.

That breaks into four questions, in order. Each one only matters if the previous one is already true — which is why the work was sequenced this way rather than starting with the exciting part.

LAYER 1 Reach — is the agent allowed to fetch the page?

A robots policy decides who may look. Ours previously allowed every AI crawler by accident, through a catch-all rule. It now names 21 of them explicitly and splits them: crawlers that *cite* you are allowed, crawlers that only *train on* you are blocked.

LAYER 2 Read — can it understand the page without running JavaScript?

Most crawlers and many agents fetch raw HTML and never execute JavaScript. Before this work every listing page returned **zero products** to them, and the storefront rendered a blank page entirely. This is where nearly all the value sits.

LAYER 3 Query — can it ask the catalogue a question instead of scraping?

Reading pages is slow, lossy and easy to get wrong. A public read-only API and an MCP server let an assistant ask directly and get typed, current data back.

LAYER 4 Transact — can it complete a purchase?

Deliberately not built. The protocols are immature and the payment stack would have to change. Worth watching, not worth speculating on.

The benefits, concretely

1. You can appear in AI answers at all

This is binary, not gradual. An agent that fetches a category page and finds no products does not try harder — it uses a competitor whose page it could read. Before this work, every listing page on all three brands returned nothing usable. Whatever share of shopping moves to assistants, the store was ineligible for all of it.

2. The facts it quotes are your facts

An assistant that cannot find a price will estimate one, and it will be wrong in public and attributed to you. Structured data removes the guesswork: price, availability, part number, condition and 24 typed specifications per product are stated in a form that cannot be misread. The MCP tools go further and instruct the model in writing never to state a price, SKU or specification that did not come from a tool result.

3. The same work is ordinary SEO — this is not a bet on AI

Server-rendered listings and schema.org markup are what traditional search engines have wanted for a decade. Google's crawler was getting the same blank page. So even if agent-driven shopping grew more slowly than expected, the work pays for itself in organic search. That is the strongest argument for having done it now: **the downside case is still a win.**

4. Agents can answer questions your own filters cannot

The catalogue runs to thousands of products with detailed specifications that were previously trapped in an on-page table — readable by a person, invisible to software. Typed as data, they become filterable: burner count, cooking area, fuel type, cutout dimensions. "Will it fit my 34-inch cutout" is now answerable without a human reading a spec sheet.

5. Traffic that arrives is further along

Someone who arrives after an assistant has already matched your product to their constraints has skipped most of the comparison stage. The visit count may be modest; the intent behind each visit is not.

6. You control the terms instead of discovering them later

The crawler policy is now an explicit decision — allow the crawlers that send traffic, block the ones that only consume product photography and copy to train models. Reversible by moving one list. Equally, the endpoints are rate-limited rather than walled off, which keeps well-behaved agents working while stopping one client from driving unbounded load.

7. No trade-off against page speed

The obvious worry with "live data for agents" is that it conflicts with caching pages for humans. It does not have to: humans get cached pages, agents get the API, and the API reads through to current data. Both audiences are served correctly by different paths.

What it is not, and what it costs

Being fair about the downsides. A public catalogue API is consumable by competitors as easily as by agents — which is why it returns only the fields an agent needs rather than whole product records, and why it is rate-limited. Structured data is also a commitment: publishing a shipping rate or return policy that contradicts what checkout actually does is worse than publishing nothing, and is exactly what gets a merchant penalised. The return policy is now published because the storefront already states it in full; the shipping rate is still withheld, because the site advertises a threshold that checkout does not necessarily charge. Finally, none of this guarantees traffic — it makes the store *eligible* for a channel it was locked out of. Eligibility is a precondition, not a result.

Part Two — How to test it, in five levels

The levels match the layers above. Run them in order: a failure at a low level makes everything above it meaningless, so there is no point testing the MCP server if the pages are still blank.

Set the target first. Run the whole sequence once **per brand** — they are three separate deployments and can differ.

```
BASE=https://your-brand-domain.com # repeat for each of the three
```

Level 0 — The one test that matters most

Open a category page in a real browser with JavaScript disabled. Chrome: Settings → Privacy and security → Site settings → JavaScript → Don't allow. Then load `$BASE/category/grills-and-smokers` .

Expected: a full page — products, prices, filters, navigation. **Before this work:** a completely blank white page.

This deserves to be first because it is the failure that hid from every other measurement. Earlier checks confirmed the product markup was present in the HTML source, and it was — but React had streamed it into a hidden element that only client-side JavaScript revealed. Every source-level test passed while the page was in fact blank to anything that does not run JavaScript. **No automated check substitutes for looking at it.**

Level 1 — Reach: is the agent allowed in?

```
# 21 named AI crawlers plus the catch-all rule -> 22
curl -s "$BASE/robots.txt" | grep -ci '^user-agent:'

# Training crawlers are blocked. Expect a "Disallow: /" under GPTBot.
curl -s "$BASE/robots.txt" | grep -A2 'GPTBot'

# Search crawlers are allowed. Expect "Allow: /" under OAI-SearchBot.
curl -s "$BASE/robots.txt" | grep -A2 'OAI-SearchBot'

# The catalogue API is carved out of the blanket /api/* block.
curl -s "$BASE/robots.txt" | grep '/api/catalog'

# Agent-facing guide, and its category list. Expect 8, not 0.
curl -s "$BASE/llms.txt" | grep -c '/category/'
curl -s "$BASE/llms-full.txt" | head -40
```

CHECK	PASS	WHAT A FAILURE MEANS
22 user-agent rules	= 22	Policy did not deploy, or an older build is live
GPTBot disallowed	Disallow: /	Training crawlers still harvesting copy and photography
OAI-SearchBot allowed	Allow: /	Serious — blocks the store from ChatGPT's shopping answers
llms.txt categories	8	The build silently dropped the category list. This exact bug shipped once

robots.txt passing is not proof the door is open. There is no Cloudflare or WAF in front of these sites, so Vercel's own bot protection is the only other gate — and if it is enabled it can challenge the very crawlers robots.txt invites, silently. Check *Project* → *Settings* → *Firewall / Bot Management* on all three projects. A robots.txt test cannot detect this.

Level 2 — Read: can it understand the page?

```
# Products visible without JavaScript. Expect 30+, was 0.
curl -s "$BASE/category/grills-and-smokers" \
  | grep -oE 'href="[^"]*/product/[^"]*"' | sort -u | wc -l

# Prices present. Note: no decimals — the grid renders "$1,499".
curl -s "$BASE/category/grills-and-smokers" \
  | grep -oE '\$[0-9,]+' | sort -u | wc -l

# Structured data blocks. Expect 2 script tags on a listing page.
curl -s "$BASE/category/grills-and-smokers" \
  | grep -o 'application/ld+json' | wc -l

# Same on a product page, plus the specification count.
curl -s "$BASE/<path-to-a-product>" | grep -o 'PropertyValue' | wc -l

# Semantic landmarks on a product page. Expect 5 labelled sections.
curl -s "$BASE/<path-to-a-product>" | grep -o '<section aria-label' | wc -l
```

Then validate the markup rather than merely counting it. Extract a JSON-LD block and paste it into **validator.schema.org**, and run the product URL through **Google's Rich Results Test**. Counting tags proves they exist; only a validator proves they are correct.

```
curl -s "$BASE/<path-to-a-product>" \
  | grep -oP '(?<=<script type="application/ld\+json">).*?(?=</script>).'
```

ROUTE	EXPECTED SCHEMA TYPES
Any page	Organization , WebSite
Category / brand / listing	+ BreadcrumbList , ItemList (30 products)
Category index	+ BreadcrumbList , ItemList (8 categories)
Product page	+ Product , BreadcrumbList , FAQPage

One rule worth enforcing forever. The products listed in `ItemList` must match the products the page actually shows. If they ever drift apart, that is cloaking — showing crawlers something different from users — and it carries a manual penalty risk, not a ranking nudge. Any change to listing filters should be re-checked against the emitted schema.

Level 3 — Query: can it ask instead of scrape?

```
# Search returns typed results with real titles and prices.
curl -s "$BASE/api/catalog/search?q=blaze&max_price=3000&limit=5" | head -40

# A handle from those results resolves to full detail.
curl -s "$BASE/api/catalog/product/<handle-from-above>" | head -40

# The machine-readable contract.
curl -s "$BASE/openapi.json" | head -20

# Rate-limit headers on every response.
curl -sI "$BASE/api/catalog/search?q=grill" | grep -i 'ratelimit'
```

Then test the paths that matter more than the happy one — an agent's behaviour when things go wrong is what separates a useful integration from one that invents answers:

REQUEST	EXPECT	WHY IT MATTERS
Unknown product handle	404	Typed error, so the agent says "not found" instead of guessing
<code>min_price</code> above <code>max_price</code>	400	Bad input is rejected clearly, not answered with nonsense
Exceed the rate limit	429	Must include <code>Retry-After</code> so a good agent backs off and returns
Any response	RateLimit-*	Lets a client pace itself <i>before</i> being refused

```
# Fire 340 concurrent requests at a 300/min bucket.
# Expect ~300 x 200 then the rest 429, with a Retry-After header.
seq 1 340 | xargs -P 40 -I{} curl -s -o /dev/null -w "%{http_code}\n" \
"$BASE/api/popular_searches" | sort | uniq -c
```

Check this before declaring Level 3 working in production. The catalogue API builds absolute product URLs from the site's configured base URL, and the MCP server calls the catalogue API through that same value. In the local environment files it is `localhost:3000` . If a production deployment inherits that, search results come back with `localhost` URLs an agent cannot follow, and the MCP tools fail outright — while the site itself looks perfectly healthy. Confirm the live deployments set it to the real domain.

Level 4 — Callable: does the MCP server work?

MCP is what lets an assistant treat the catalogue as a native tool. Four requests establish that it works end to end:

```
# 1. Describe the endpoint – a plain GET, no protocol needed.
curl -s "$BASE/api/mcp"

# 2. Handshake. Expect protocolVersion "2025-06-18".
curl -s -X POST "$BASE/api/mcp" -H 'Content-Type: application/json' \
-d '{"jsonrpc":"2.0","id":1,"method":"initialize"}'

# 3. Tool list. Expect search_products and get_product.
curl -s -X POST "$BASE/api/mcp" -H 'Content-Type: application/json' \
-d '{"jsonrpc":"2.0","id":2,"method":"tools/list"}'

# 4. A real question, answered by a tool call.
curl -s -X POST "$BASE/api/mcp" -H 'Content-Type: application/json' \
-d '{"jsonrpc":"2.0","id":3,"method":"tools/call","params":{"name":"search_products",
"arguments":{"q":"4 burner built-in gas grill","max_price":3000}}}'

# 5. Failure paths. Expect isError:true, and error -32601.
curl -s -X POST "$BASE/api/mcp" -H 'Content-Type: application/json' \
-d '{"jsonrpc":"2.0","id":4,"method":"tools/call","params":{"name":"get_product","arguments":{"handle":"does-not-exist}}}'
curl -s -X POST "$BASE/api/mcp" -H 'Content-Type: application/json' \
-d '{"jsonrpc":"2.0","id":5,"method":"nope"}'
```

The failure cases are the point. A tool that fails *silently* lets a model fill the gap with invention; these return an explicit error plus an instruction to tell the user the lookup failed.

Level 5 — The end-to-end test: ask an actual assistant

Every level above tests a mechanism. This one tests the outcome, and it is the only test whose result a non-technical stakeholder can judge directly.

1. **Attach the MCP server** to an assistant that supports custom connectors, pointed at `$BASE/api/mcp`. Ask a buying question with real constraints — budget, burner count, a cutout width. Confirm the answer cites actual products at actual prices, and spot-check two against the site.
2. **Without any connector**, ask ChatGPT, Perplexity, Claude and Google's AI results a question your catalogue should win — a specific brand and model, then a broader "best X under \$Y". Record whether the store is cited, whether the facts are right, and who is cited instead.
3. **Paste a category URL** into an assistant with browsing and ask it to list the products and prices. It should succeed. Before this work it would have described an empty page.

Set expectations on timing. Levels 0–4 are true the moment a deploy completes. Level 5 is not: crawlers have to re-fetch and re-index, which takes weeks, and assistants that rely on a search index inherit that delay. Judging this in days will produce a false negative. Record a baseline now, before broad re-crawling, so there is something to compare against.

Traps that make a passing test lie

Each of these produced a wrong answer during this work. They are listed because they will do it again to the next person who tests.

TRAP	WHAT GOES WRONG	DO THIS INSTEAD
"It's in the HTML source"	Markup can be present and still invisible — this is precisely how a blank storefront passed every check for months	Load the page in a browser with JavaScript off. Always.
<code>grep -c</code>	Counts matching <i>lines</i> , not occurrences. Minified HTML is one long line, so three blocks report as "1". Earlier notes in this project were wrong for this reason	<code>grep -o pattern wc -l</code>
Testing in dev only	The dev server renders every request fresh, so build-time caching bugs are invisible. A category list vanished from a production file while dev looked perfect	Test a real build, then test production
Over-strict patterns	A price regex requiring decimals reported "no prices" on a page rendering <code>\$1,499</code>	Eyeball the raw output before trusting a count of zero
Errors that are caught and hidden	Several functions catch their own failures and return empty. The page renders, the log stays clean, the data is simply missing	Assert on expected <i>counts</i> , never on "no error"
Testing one brand	Three separate deployments with separate settings. One brand was down for days on a fault the other two carried but never triggered	Run the full sequence three times
Trusting a green build	A build can exit 0 while emitting warnings that pages shipped with empty data	Read the build log; treat static-rendering warnings as failures

Is it worth it? — what to measure over the next months

Levels 0–4 answer "is it working". This answers "was it worth doing", which is a slower and genuinely harder question — agent traffic often arrives without a referrer, so clean attribution is not always available. Use several signals rather than looking for one number.

SIGNAL	WHERE	WHAT GOOD LOOKS LIKE
AI crawler visits	Server / Vercel logs, filtered by user-agent — <code>OAI-SearchBot</code> , <code>PerplexityBot</code> , <code>Claude-SearchBot</code> , <code>ChatGPT-User</code>	Rising, and fetching listing pages rather than only the homepage. The most direct evidence agents can now use the site
Citation checks	Manual, monthly — the same five buying questions asked of each major assistant	The store appears where it previously did not. Crude, but it measures the actual outcome and needs no attribution
Rich results	Google Search Console → Enhancements (Products, Breadcrumbs, FAQ)	Valid items climbing toward catalogue size, errors near zero
Listing-page search performance	Search Console, category and brand pages specifically	Impressions and average position improving — these pages had no readable product content before
API and MCP usage	Request counts on <code>/api/catalog/*</code> and <code>/api/mcp</code>	Any sustained non-zero volume is a channel that did not exist
Referral sessions	Analytics, referrers containing <code>chatgpt</code> , <code>perplexity</code> , <code>claude</code> , <code>copilot</code>	Growing. Undercounts real influence, so treat it as a floor, not the whole picture

The honest bottom line. The clearest return here is not a forecast about AI shopping — it is that every listing page on all three brands was returning zero products to every non-JavaScript client, search engines included, and the storefront rendered blank. That was costing traditional organic search before agents entered the picture. The agentic work paid for fixing it, and the AI channel is upside on top of a fix that was overdue on its own merits.

Reference. Per-brand release steps and rollback: `docs/deploy-agentic-ai-2026-08-07.md` . Engineering detail and open decisions: `docs/agentic-ai-readiness.md` . Full status report: `AGENTIC-AI-READINESS-REPORT.pdf` . This week's changes: `docs/development-update-2026-08-11.pdf` .